

Government College of Engineering, Jalgaon
(An Autonomous Institute of Government of Maharashtra)

Name :	PRN :
Class : L. Y. B.Tech Computer	Batch:
Subject : CO456U Cloud Computing Lab	Sem : VIII th
Course Teacher: Prof. S. D. Cheke	Academic Year : 2025-26
Date of Performance :	Date of Completion :

Practical No. 7

Aim: Create and deploy lambda function that takes request and put that data to dynamodb.

Theory :

Objective: To create an AWS Lambda function that receives a request and stores the data into a DynamoDB table.

What is AWS Lambda?

AWS Lambda is a serverless computing service that allows you to run code without managing servers.

- Executes code on demand
- Automatically scales
- Charges only for execution time

What is DynamoDB?

Amazon DynamoDB is a fully managed NoSQL database service by AWS.

- Stores data in key-value format
- Highly scalable and fast
- No server management required

What is Serverless Architecture?

Serverless Architecture is a model where the cloud provider manages infrastructure, and developers focus only on code.

How It Works

1. User sends request (via API Gateway or test event)
2. Lambda function processes request
3. Data is inserted into DynamoDB table

Requirements

- AWS Account
- IAM Role with DynamoDB access
- Basic knowledge of Python

Procedure / Steps

Step 1: Open DynamoDB

1. Go to AWS Console
2. Search for DynamoDB
3. Click **Create Table**

Step 2: Configure Table

- **Table Name:** users
- **Primary Key:** user_id (String)

Click **Create Table**

DynamoDB > Tables > Create table

Create table

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name
This will be used to identify your table.

users

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key
The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

user_id

String

1 to 255 characters and case sensitive.

Sort key - optional
You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

Enter the sort key name

String

1 to 255 characters and case sensitive.

Tables (1) [Info](#)

Last updated
April 15, 2026, 12:18 (UTC+5:30)

Actions

Delete

Create table

Find tables

Filter by tag
Any tag key

Filter by tag value
Any tag value

< 1 >

☐	Name ▲	Status ▼	Partition key ▼	Sort key ▼	Indexes ▼	Replication Regions ▼	Deletion protection ▼	Favorite ▼	Read capacity mode ▼	Write capacity
☐	users	Active	user_id (S)	-	0	0	Off		On-demand	On-demand

Step 2: Create IAM Role

1. Go to IAM → Roles
2. Click **Create Role**
3. Select:
 1. Trusted entity: **AWS Service**
 2. Use case: **Lambda**

2 | Page

Select trusted entity [Info](#)

Trusted entity type

☒ **AWS service**
Allow AWS services like EC2, Lambda, or others to perform actions in this account.

☐ **AWS account**
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.

☐ **Web identity**
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.

☐ **SAML 2.0 federation**
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.

☐ **Custom trust policy**
Create a custom trust policy to enable others to perform actions in this account.

Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case
Lambda

Choose a use case for the specified service.

Use case

☒ **Lambda**
Allows Lambda functions to call AWS services on your behalf.

☐ **AWSServiceRoleForLambda**
Allows Lambda to manage AWS resources on your behalf.

Cancel **Next**

4. Attach policy:
 1. **AmazonDynamoDBFullAccess**

Add permissions [Info](#)

Permissions policies (1/1144) [Info](#)

Choose one or more policies to attach to your new role.

Q AmazonDynamoDBFullAccess X

Filter by Type
All types 3 matches

Policy name	Type	Description
☒ AmazonDynamoDBFullAccess	AWS managed	Provides full access to Ama
☐ AmazonDynamoDBFullAccess_v2	AWS managed	Provides full access to Ama
☐ AmazonDynamoDBFullAccesswithDataPipeline	AWS managed	This policy is on a deprecate

► Set permissions boundary - optional

5. Name: lambda-dynamodb-role
6. Click **Create Role**

Roles (4) [Info](#)

Delete Create role

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Q Search

< 1 >

Role name	Trusted entities	Last activity
☐ AWSServiceRoleForResourceExplorer	AWS Service: resource-explorer-2 (Se	2 hours ago
☐ AWSServiceRoleForSupport	AWS Service: support (Service-Linker	-
☐ AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service	-
☐ lambda-dynamodb-role	AWS Service: lambda	22 minutes ago

Step 3: Open Lambda

1. Go to AWS Lambda
2. Click **Create Function**

Step 4: Configure Function

- **Function Name:** insertUser
- **Runtime:** Python 3.x
- **Execution Role:** Create new role with DynamoDB full access

Function name
Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyph

Runtime | [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Durable execution - new | [Info](#)
Enable durable execution to simplify building resilient multi-step applications that checkpoint progress and resume after interruptions. Su
☐ Enable

Architecture | [Info](#)
Choose the instruction set architecture you want for your function code.
☐ arm64
☒ x86_64

Permissions [Info](#)
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can

▼ Change default execution role
Execution role
To access other AWS services and resources your function needs an IAM role. Choose a role with the right permissions for your function.
☐ Create default role ☒ Use another role
 [View role details in IAM](#) [↗](#) [↻](#) [Create new role](#)

Write Lambda Code

```
import json

import boto3

dynamodb = boto3.resource('dynamodb')

table = dynamodb.Table('Users')

def lambda_handler(event, context):

    try:

        # Get data from request
```

```

user_id = event['user_id']

name = event['name']

# Insert into DynamoDB

table.put_item(

    Item={

        'user_id': user_id,

        'name': name

    }

)

return {

    'statusCode': 200,

    'body': json.dumps('Data inserted successfully')

}

except Exception as e:

    return {

        'statusCode': 500,

        'body': str(e)

    }

```

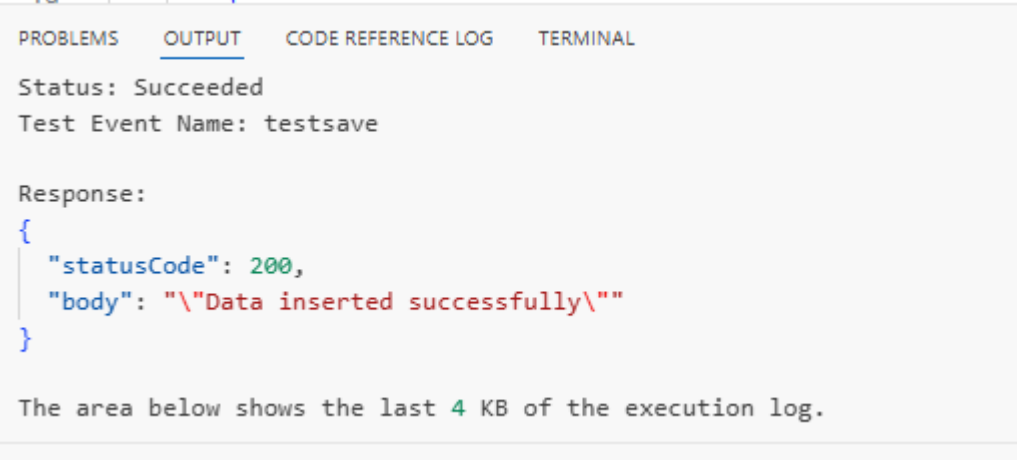
Create Test Event

1. Click **Test**
2. Configure test event:

```

{
  "user_id": "101",
  "name": "John Smith"
}

```



```

PROBLEMS  OUTPUT  CODE REFERENCE LOG  TERMINAL

Status: Succeeded
Test Event Name: testsave

Response:
{
  "statusCode": 200,
  "body": "\"Data inserted successfully\""
}

The area below shows the last 4 KB of the execution log.

```

Table: users - Items returned (1)

Scan started on April 15, 2026, 12:33:40

Actions

Create item

☐	user_id (String)	▼	name	▼
☐	101		John Smith	

Conclusion:

In this way, we have successfully implemented and completed the creation and deployment of a Lambda function that stores request data into DynamoDB.

Prof. S. D. Cheke
Course Teacher